

Python libraries: Installation tips and tricks

Python offers various methods for installing libraries, each tailored to different needs and preferences. The following table outlines various helpful techniques and strategies that can be used during library installation.

Summary	Details	Examples
Using <i>pip</i>	<i>pip</i> is the default package manager for Python, and it simplifies the process of installing libraries	<i>pip install matplotlib</i>
Using <i>conda</i>	<i>conda</i> is a package manager primarily used for managing packages in data science and scientific computing environments	<i>conda install matplotlib</i>
Particular version	Using a particular version while installing	<i>pip install torch==0.2.0_4</i> <i>conda install pytorch==1.0.</i>
Version check	Checking if any version of the libraries exists	<i>pip show matplotlib</i>
Upgrade existing packages ('-U' or '--upgrade')	To upgrade an existing package to the latest version, you can use the '-U' or '--upgrade' option	<i>pip install -U matplotlib</i>
Quiet installation ('-q' or '--quiet')	If you prefer a quieter installation process without verbose output, you can use the '-q' or '--quiet' option	<i>pip install -q bokeh</i>
Ignoring dependencies ('--no-deps')	In certain cases, you may want to install a library without installing its dependencies	<i>pip install --no-deps bokeh</i>
<i>requirements.txt</i>	You can install multiple packages listed in a requirements file using the '-r' or '--requirement' option	<i>pip install -r requirements.txt</i>

an interactive development environment with a read-eval-print loop (REPL). This feature enables developers to execute Python code interactively, debug programs in real-time, and experiment with hardware peripherals without the need for compiling and flashing firmware repeatedly.

languages like C, MicroPython may have slower execution speeds and higher memory usage, particularly in performance-critical applications.

- The MicroPython ecosystem may not be as mature or comprehensive as other embedded development ecosystems, leading to potential

limitations in terms of available libraries, tools, and community support.

Overall, Python's future looks bright, driven by its versatility, ease of use, and strong community support. Python as a language looks promising, with continued growth and adoption across various domains. **END** 🐧

Limitations

- While MicroPython supports a wide range of microcontroller platforms, some hardware-specific features and peripherals may not be fully supported or optimised in certain implementations.
- Compared to native compiled

By: Dibyendu Banerjee and Sourav Kairi

Dibyendu Banerjee is a visionary leader driving transformative initiatives as a Sr. Architect within Cognizant's AI and Analytics practice. He is a certified Deep Learning and NLP practitioner with 18+ years of IT experience. His areas of expertise are in Python, R, Java, Google Cloud Platform and emerging open source technologies.

Sourav Kairi is a programming analyst at Cognizant. His areas of interest include Generative AI, Prompt Engineering and LLM.